🎯 Atividade Prática: Introdução à Linguagem C - Sintaxe e Boas Práticas de Programação

📌 **Objetivo:** Introduzir os alunos à **linguagem C**, com foco na **sintaxe básica**, **entrada e saída de dados**, **modificadores de acesso** e **boas práticas de programação**. A atividade foi estruturada com base na metodologia SENAI de ensino profissionalizante, para que os alunos aprendam os fundamentos e apliquem esses conceitos em exercícios práticos, reforçando as boas práticas de programação.

📌 **Formato: Atividade Individual**
📌 **Duração: 2 horas**
📌 **Ferramentas: Compilador de C** (como Dev C++, Code::Blocks, ou IDE online como replit.com)
📌 **Entrega:** Código completo, com explicações sobre a estrutura do programa, boas práticas e o que foi aprendido.

📖 Contextualização e Situação Problema:

Você foi designado para desenvolver um programa de **controle de estoques** para uma pequena loja de produtos. O programa precisa realizar as seguintes funções básicas:

1. **Entrada de dados** do produto (nome, preço e quantidade).
2. **Saída de dados** do produto (exibir as informações inseridas).
3. **Usar modificadores de acesso** para controlar a visibilidade das variáveis dentro do código.

O objetivo é aprender a **estruturar um programa em C**, compreendendo as **boas práticas de programação** e os **conceitos fundamentais** da linguagem, como declaração de variáveis, funções, modificadores de acesso e como trabalhar com entrada e saída de dados.

📋 Passo a Passo de Execução:

1. Preparação do Ambiente (10 minutos):

- Abra seu **compilador de C** (Dev C++, Code::Blocks, ou IDE online).

- Crie um **novo projeto** ou **novo arquivo** de código em C.
- Salve o arquivo com o nome `controle_estoque.c`.

2. Estrutura Básica do Programa (20 minutos):

Inicie com a estrutura básica de um programa em C, incluindo as bibliotecas necessárias e a função principal (`main`). O exemplo abaixo mostra como isso deve ser feito:

```C/C++
#include <stdio.h>  // Biblioteca para entrada e saída de dados

// Função principal onde o programa começa a execução
int main() {
    // Declaração de variáveis
    char nomeProduto[50];  // Nome do produto
    float precoProduto;    // Preço do produto
    int quantidadeProduto; // Quantidade de produtos

    // Entrada de dados
    printf("Digite o nome do produto: ");
    scanf("%s", nomeProduto); // Lê o nome do produto

    printf("Digite o preço do produto: ");
    scanf("%f", &precoProduto); // Lê o preço do produto

    printf("Digite a quantidade do produto: ");
    scanf("%d", &quantidadeProduto); // Lê a quantidade do produto

    // Exibição dos dados inseridos
    printf("\nProduto: %s\n", nomeProduto);
    printf("Preço: R$ %.2f\n", precoProduto);
    printf("Quantidade: %d\n", quantidadeProduto);

    return 0;
}
```

Explicação do Código:

1. **Biblioteca `stdio.h`:** Essa biblioteca é necessária para usar as funções de entrada e saída de dados, como `printf` (para exibição) e `scanf` (para leitura de dados).

2. **Função `main`:** Toda execução de um programa em C começa pela função `main()`.

3. **Declaração de Variáveis:**

    - `char nomeProduto[50];` para armazenar o nome do produto (máximo de 50 caracteres).

    - `float precoProduto;` para armazenar o preço do produto.

    - `int quantidadeProduto;` para armazenar a quantidade do produto.

4. **Entrada de Dados:**

    - Usamos `scanf` para ler os dados digitados pelo usuário.

5. **Exibição dos Dados:**

    - Usamos `printf` para exibir as informações que o usuário inseriu.

3. Boas Práticas de Programação (20 minutos):

Agora que você tem o esqueleto do programa, vamos adicionar algumas **boas práticas** de programação.

- **Comente o código:** A cada parte importante do código, adicione um comentário explicando o que ele faz.

- **Usar nomes significativos para variáveis:** Certifique-se de que os nomes das variáveis reflitam o seu conteúdo e a sua função no programa.

- **Indentação:** Mantenha a indentação correta do código para melhorar a legibilidade.

```C/C++
#include <stdio.h>  // Biblioteca para entrada e saída de dados

// Função principal onde o programa começa a execução
int main() {
```

```
    // Declaração de variáveis
    char nomeProduto[50];  // Nome do produto
    float precoProduto;    // Preço do produto
    int quantidadeProduto; // Quantidade de produtos

    // Entrada de dados
    printf("Digite o nome do produto: ");
    scanf("%s", nomeProduto); // Lê o nome do produto

    printf("Digite o preço do produto: ");
    scanf("%f", &precoProduto); // Lê o preço do produto

    printf("Digite a quantidade do produto: ");
    scanf("%d", &quantidadeProduto); // Lê a quantidade do produto

    // Exibição dos dados inseridos
    printf("\nProduto: %s\n", nomeProduto);
    printf("Preço: R$ %.2f\n", precoProduto);
    printf("Quantidade: %d\n", quantidadeProduto);

    return 0;
}
```

4. Modificadores de Acesso (20 minutos):

Para entender os **modificadores de acesso** em C, podemos simular o uso de variáveis **globais** e **locais**. Em um programa de tamanho maior, as variáveis globais podem ser acessadas de qualquer parte do código, enquanto as variáveis locais só são acessíveis dentro da função em que são declaradas.

Modificador de Acesso (Global vs Local):

```
C/C++
#include <stdio.h>

int precoProdutoGlobal;  // Variável global, pode ser acessada por
qualquer função
```

```
// Função principal onde o programa começa a execução
int main() {
    // Variáveis locais
    char nomeProduto[50];
    int quantidadeProduto;

    // Acessando a variável global
    precoProdutoGlobal = 25;  // Definindo o valor da variável
global

    // Entrada de dados
    printf("Digite o nome do produto: ");
    scanf("%s", nomeProduto);

    printf("Digite a quantidade do produto: ");
    scanf("%d", &quantidadeProduto);

    // Exibição dos dados
    printf("\nProduto: %s\n", nomeProduto);
    printf("Preço Global: R$ %d\n", precoProdutoGlobal);  //
Exibindo a variável global
    printf("Quantidade: %d\n", quantidadeProduto);

    return 0;
}
```

Explicação:

- **Variável global:** `precoProdutoGlobal` foi declarada fora da função `main()`, portanto, pode ser acessada em qualquer parte do programa.

- **Variáveis locais:** As variáveis `nomeProduto` e `quantidadeProduto` são locais e podem ser acessadas apenas dentro da função `main()`.

5. Teste do Programa e Aprendizado (20 minutos):

Agora que você terminou o código, execute o programa e teste com diferentes entradas. Verifique se ele está funcionando conforme o esperado e **registre os dados** (nome do produto, preço e quantidade).

1. **Entrada de dados:** Insira informações de diferentes produtos e verifique se o programa exibe as informações corretamente.

2. **Saída de dados:** Observe como os dados são apresentados na tela.

📄 Entrega da Atividade:

O aluno deve entregar:

1. **Código completo** com a **entrada e saída de dados**.

2. **Comentário e explicação** sobre as **boas práticas de programação** utilizadas.

3. **Resultados obtidos** ao testar o programa com diferentes entradas.

📋 Rubrica de Avaliação:

Critério	Descrição	Peso (%)
Correção do Código	O código executa corretamente a entrada e saída de dados e utiliza modificadores de acesso	40%
Boas Práticas de Programação	O código segue boas práticas de programação (indentação, comentários e nomenclatura)	30%
Organização e Clareza	O código está bem organizado, com boa legibilidade e clareza	20%
Testes e Resultados	O programa foi testado adequadamente e produziu os resultados esperados	10%

🎯 Resumo da Atividade:

- **Objetivo:** Ensinar a sintaxe básica de **C**, boas práticas de **programação** e introdução a **entrada e saída de dados**.

- **Conceitos:** Uso de **modificadores de acesso**, **funções** e **variáveis**.

- **Ferramenta:** Utilização de **smartphones** para escrever, executar e testar códigos.

Essa atividade permitirá que os alunos desenvolvam as habilidades essenciais de **programação** e compreendam a importância de **boas práticas** para escrever código legível e eficiente.